

Comando /goal

Trabalho autônomo até uma condição de sucesso

Resumo técnico sobre sintaxe, funcionamento interno, comparação com outros modos, limitações e disponibilidade do comando /goal.

O que é o comando /goal

O **/goal** define uma condição de conclusão e instrui o Claude a continuar trabalhando autonomamente em direção a essa condição, sem exigir que você intervenha a cada etapa. Depois de cada turno, um modelo pequeno e rápido (o **avaliador**) verifica se a condição já foi atingida. Se ainda não, o Claude inicia outro turno automaticamente em vez de devolver o controle para você.

Principais casos de uso:

- Trabalho substancial com um estado final verificável.
- Tarefas que exigem vários turnos para concluir (ex.: migrar um módulo para uma nova API até todos os pontos de chamada compilarem e os testes passarem).
- Implementar um design doc até que todos os critérios de aceite sejam atendidos.
- Dividir arquivos grandes em módulos focados até cada um respeitar um limite de tamanho.
- Trabalhar em uma fila de issues rotuladas até a fila esvaziar.

02 Sintaxe e uso

Definir um objetivo:

```
/goal <condição>
```

Exemplo:

```
/goal all tests in test/auth pass and the lint step is clean
```

Consultar o objetivo ativo (sem argumentos):

```
/goal
```

Limpar um objetivo ativo:

```
/goal clear  
# aliases aceitos: stop, off, reset, none, cancel
```

Uso não interativo:

```
claude -p "/goal <condição>"  
  
# com saída detalhada:  
claude -p "/goal <condição>" --output-format stream-json --verbose
```

Limite de tamanho da condição: até **4.000 caracteres**.

03 Como funciona internamente

1. Ativação — rodar `/goal <condição>` inicia um turno imediatamente, usando a condição como diretiva.

2. Loop de avaliação — ao final de cada turno:

- O Claude Code envia a condição e toda a conversa até então para o modelo pequeno e rápido configurado (padrão: Haiku, na API da Anthropic).
- O avaliador devolve um de três vereditos, com justificativa:
 - **Ainda não atingida** → o Claude continua, usando a justificativa como orientação para o próximo turno.
 - **Atingida** → o objetivo é encerrado automaticamente; registrado como "achieved" na transcrição.
 - **Impossível** → o modelo julgou que a condição nunca poderá ser satisfeita; o objetivo é encerrado e marcado como "failed".

3. Condições de parada:

- O modelo confirma que a condição foi atingida.
- O modelo julga a condição impossível.
- Um turno falha por um erro que precisa ser corrigido manualmente (encerra o objetivo na hora).
- Você roda `/goal clear` manualmente.
- Outros erros entram em nova tentativa ou pausam o objetivo (ex.: rate limits).

4. Trabalho em segundo plano:

- Se um subagente ou comando de shell em segundo plano ainda está rodando quando um turno termina, a avaliação é adiada para o próximo turno que terminar sem trabalho pendente.
- Check-ins ocorrem a cada 30 minutos (configurável) enquanto o objetivo aguarda trabalho em segundo plano.

5. Retomada de sessão:

Ao retomar uma sessão com um objetivo ativo, o Claude Code restaura a condição, mas reinicia a contagem de turnos, o cronômetro e a linha de base de consumo de tokens.

04 Exemplos reais da documentação

```
/goal all tests in test/auth pass and the lint step is clean
```

```
/goal CHANGELOG.md has an entry for every PR merged this week
```

```
/goal every call site to the old API has been migrated to the  
new one, all tests pass, and no references to the old module  
remain in the codebase
```

```
# com limite de turnos:
```

```
/goal implement user authentication or stop after 20 turns
```

05 Diferenças em relação a outros modos

Recurso	/goal	/loop	Stop Hook	Auto Mode sozinho
Próximo turno começa quando	Turno anterior termina (ou check-in de espera)	Intervalo de tempo passa	Turno anterior termina	Nunca (só aprova 1 turno)
Para quando	Condição atingida/impossível, limpeza manual, ou erro a corrigir	Você para manualmente	Seu script/prompt decide	Claude julga o trabalho concluído
Escopo	Só a sessão atual	Só a sessão atual	Aplicado a toda sessão (no escopo)	—
Avaliação	Baseada em modelo (avaliador dedicado)	N/A	Baseada em modelo ou script	N/A

Diferença-chave em relação a um prompt comum:

Um prompt normal pede para o Claude fazer algo uma vez. O **/goal** envolve a condição em um Stop hook baseado em prompt, restrito à sessão, fazendo o Claude checar após **cada** turno se deve continuar ou parar — independente do próprio julgamento do Claude sobre se a tarefa está concluída.

Diferença-chave em relação ao Auto Mode:

O Auto Mode remove prompts de aprovação por ferramenta dentro de um único turno. O **/goal** remove prompts por turno, adicionando um avaliador separado que decide a conclusão com base na sua condição (e não no julgamento do Claude).

06 Limitações e configurações

Limitações:

- Um objetivo por sessão — definir um novo objetivo substitui o anterior.
- O avaliador não roda comandos nem lê arquivos por conta própria — só julga o que o Claude já expôs na conversa.
- A condição precisa ser verificável a partir da saída do Claude na transcrição.
- Se o Claude responder ao avaliador sem progredir (sem uso de ferramentas por vários turnos), o Claude Code interrompe o loop e devolve o controle para você.

Configurações disponíveis (variáveis de ambiente):

Variável	Finalidade	Padrão
ANTHROPIC_DEFAULT_HAIKU_MODEL	Troca o modelo usado como avaliador	Haiku (varia por provedor)
CLAUDE_CODE_GOAL_CHECKIN_MINUTES	Intervalo de check-in para trabalho em segundo plano longo	30 min; 0 desativa check-ins e retries automáticos

07 Permissões e confiança de workspace

- O /goal segue a mesma regra de confiança de pasta de trabalho aplicada aos hooks nos arquivos de configuração.
- Fica indisponível se `disableAllHooks` estiver `true` nas configurações, ou se `allowManagedHooksOnly` estiver definido nas configurações gerenciadas.
- Para rodar um objetivo em modo não interativo numa pasta não confiável, é preciso aprovar interativamente antes — caso contrário a execução é recusada.
- O /goal não muda seu modo de permissão: no Auto Mode os turnos do objetivo rodam sem supervisão (recomendado); no modo manual, o Claude continua pedindo aprovação para chamadas de ferramenta fora das suas configurações.

08 Disponibilidade por plataforma e versão

- Plataformas: CLI do Claude Code, app Desktop, extensão do VS Code, Remote Control e modo não interativo (flag `-p`).
- Requer confiança da pasta de trabalho (mesma regra dos hooks) e um modelo pequeno e rápido configurado para avaliação.
- Disponível na API da Anthropic (avaliador padrão: Haiku) e em provedores terceiros (Bedrock, Vertex AI etc. — consulte a página do provedor para o modelo Haiku padrão).
- Sem restrição de plano — disponível para todos os usuários do Claude Code.
- Versão mínima do Claude Code: **v2.1.234** (funcionalidade básica).
- Check-ins de espera: requer **v2.1.236+**.
- Retries automáticos e lógica de pausa com avisos detalhados: requer **v2.1.269+**.
- Restauração de sessão em todas as rotas de retomada (inclusive pelo seletor): requer **v2.1.239+**.

Principais pontos

- ✓ /goal define uma condição de sucesso e faz o Claude continuar trabalhando sozinho até atingi-la.
- ✓ Um avaliador (modelo pequeno e rápido) checa a condição após cada turno — atingida, impossível, ou segue em frente.
- ✓ Sintaxe: /goal <condição> para definir, /goal para consultar, /goal clear para encerrar.
- ✓ Diferente de um prompt comum: o Claude para de decidir sozinho quando parar — quem decide é o avaliador.
- ✓ Combina bem com o Auto Mode para execução totalmente autônoma, sem prompts por turno.
- ✓ Uma sessão só mantém um objetivo por vez; novo /goal substitui o anterior.
- ✓ Disponível para todos os planos, sem custo adicional — requer só confiança da pasta de trabalho.
- ✓ Versão mínima recomendada: v2.1.269+, para ter check-ins, retries e avisos completos.

Fonte oficial: code.claude.com/docs/en/goal.md — documentação relacionada: [scheduled-tasks.md \(/loop\)](#), [hooks-guide.md](#) (detalhes do Stop hook), [env-vars.md](#) (variáveis de ambiente) e [model-config.md](#) (configuração de modelos).